Soluciones - Criterio 2.d (Conversiones entre tipos de datos)

Resueltas con la teoria del tema 2.4 (coercion implicita/explicita, Number()/String()/Boolean(), regla de los 8 valores falsy, y el peligro de == frente a ===).

Ejercicio 1. El enigma de la igualdad con booleanos

if ("0" == false) { console.log("A se cumple"); } // Se cumple

if ("0") { console.log("B se cumple"); } // Se cumple

Ambos se cumplen, pero por dos motivos distintos:

- Caso A ("0" == false): el operador == es igualdad abstracta y fuerza la conversion antes de comparar. Convierte ambos operandos a numero: false -> 0 y "0" -> 0. Como 0 == 0, el resultado es true. (Es el mismo fenomeno que "" == 0 // true o [] == 0 // true.)

- Caso B (if ("0")): una estructura if aplica coercion a booleano (regla falsy/truthy). La cadena "0" no esta vacia, asi que es un valor truthy -> true. (Los unicos falsy son false, 0, -0, 0n, "", null, undefined, NaN.)

Clave: == compara convirtiendo a numero (aqui 0 == 0), mientras que if convierte a booleano (cadena no vacia -> true). Por eso el mismo texto "0" produce true en ambas, pero por caminos diferentes.

Ejercicio 2. Laboratorio. Inspector de conversiones y tipado debil

Archivos creados (sin modificar HTML ni CSS, como exige el enunciado):

- estilos.css: hoja externa tal cual la facilita el material.

- laboratorio_conversiones.html: estructura HTML dada (boton btnAnalizar, campo txtEntrada, salida pantallaSalida).

- laboratorio_conversiones.js: toda la logica, implementando los 6 hitos.

Resumen de los hitos implementados:

Hito 1: Localiza btnAnalizar, txtEntrada, pantallaSalida; addEventListener("click", analizarCoercion); normaliza con try { JSON.parse() } catch, resolviendo "undefined", "NaN" y cadenas puras.

Hito 2: Number(valorReal), String(valorReal), Boolean(valorReal) + typeof valorReal.

Hito 3: Anomalias: si el texto es "0" y Boolean(x) === true -> aviso "«0» es TRUTHY"; si es [] y Number(x) === 0 -> aviso "la matriz coerciona a 0".

Hito 4: valorReal + 10 y valorReal - 10 con su typeof, mostrando cuando + concatena y - fuerza numero.

Hito 5: Number.isNaN(Number(valorReal)) -> aviso rojo "Conversion numerica fallida: No es un numero valido".

Hito 6: Informe inyectado en #pantallaSalida con , y <br>.

Verificacion con la tabla de casos del PDF:

Entrada: 0

- Number(x): 0

- Boolean(x): false

- x + 10 / x - 10: 10 / -10

- Alerta: ninguna

Entrada: "0"

- Number(x): 0

- Boolean(x): true

- x + 10 / x - 10: "010" / -10

- Alerta: "0" es TRUTHY

Entrada: false

- Number(x): 0

- Boolean(x): false

- x + 10 / x - 10: 10 / -10

- Alerta: ninguna

Entrada: null

- Number(x): 0

- Boolean(x): false

- x + 10 / x - 10: 10 / -10

- Alerta: ninguna

Entrada: undefined

- Number(x): NaN

- Boolean(x): false

- x + 10 / x - 10: NaN / NaN

- Alerta: Number.isNaN()

Entrada: []

- Number(x): 0

- Boolean(x): true

- x + 10 / x - 10: "10" / -10

- Alerta: matriz -> 0

Entrada: "texto"

- Number(x): NaN

- Boolean(x): true

- x + 10 / x - 10: "texto10" / NaN

- Alerta: Number.isNaN()